Secure ML Governance and Zero-Downtime Deployment for Census Income Prediction

Parth Ubhe (PRN: 23070126086) and Piyush Gawade (PRN: 23070126090)
Symbiosis Institute of Technology, Pune, India
Faculty Guide: Prof. Mayur Gaikwad

***Abstract*—Operational machine learning requires controls beyond predictive accuracy: candidate releases must be evaluated, inference inputs validated, deployment exposure limited, and decisions traced to model and data versions. This project implements a post-training governance platform for the UCI Adult Census Income dataset using Flask and scikit-learn. Logistic regression serves as the production baseline and a random forest as the candidate. The platform combines authenticated inference, strict input validation, integrity checks, quality and demographic-parity gates, weighted canary routing, asynchronous shadow evaluation, automated rollback, SQLite audit logging, drift monitoring, and MLflow lineage. An official training-file split supplies 26,048 fitting and 6,513 validation records; the separate 16,281-record official test file is reserved for final evaluation. The candidate achieves 86.57% validation accuracy and 86.32% official-test accuracy, compared with 85.57% and 85.11% for the baseline. Its validation demographic-parity gap is 0.1762 under a configurable 0.20 admission limit. Forty-four automated tests validate model, security, and integration behavior. Results demonstrate a reproducible local governance workflow; they do not establish production availability, comprehensive security, or absence of demographic bias.**

***Index Terms*—MLOps, model governance, Adult Census Income, canary deployment, shadow deployment, fairness, drift monitoring, MLflow.**

I. INTRODUCTION

Deploying a classifier introduces operational questions that a test-set score alone cannot answer. A service must distinguish valid input from malformed requests, identify the exact model responsible for a prediction, limit the exposure of a new release, and recover when candidate behavior degrades. Changes in the incoming population also require monitoring even when the application remains available. These responsibilities motivate a governance layer around the inference lifecycle.

The objective of this project is to implement and evaluate that layer for census-income classification. The prediction task is whether an individual's recorded annual income exceeds USD 50,000. The selected dataset is UCI Adult [1]. The contribution is the integration of release admission, secure serving, controlled traffic routing, traceability, and monitoring into a runnable academic platform. It is a systems implementation rather than a new classification algorithm.

The design separates offline evaluation from online operation. Offline checks decide whether a candidate meets a configured policy. Online controls authenticate and validate requests, choose an admitted model, record outcomes, and watch operational degradation. Existing production and candidate pipelines remain loaded while traffic weights change, demonstrating a service-level mechanism for switching without restarting the API.

II. BACKGROUND AND DESIGN OBJECTIVES

Sculley et al. identify data dependencies, configuration, feedback loops, and system interactions as sources of technical debt in machine learning systems [2]. This project addresses a limited operational subset through explicit schemas, versioned artifacts, admission policies, and request-level audit records. Scikit-learn supplies the estimator and preprocessing components [3]; the governance logic is implemented around those components.

The principal objectives are to preserve a reproducible data-to-model lineage, reject invalid inference records, quantify candidate quality before exposure, support reversible traffic changes, and expose interpretable operational evidence. Canary releases send a configured fraction of requests to a candidate. Shadow releases instead duplicate selected inputs to a candidate while returning production predictions. These modes serve different purposes and are evaluated separately.

Adult provides a useful mixed-type classification case, but its historical origin and demographic structure restrict external validity. Ding et al. discuss limitations of using Adult as a general benchmark for fairness [4]. Accordingly, this work treats income prediction and demographic parity as an educational demonstration, not an eligibility, lending, employment, or policy decision system.

III. DATASET AND EXPERIMENTAL PROTOCOL

A. Source, Features, and Target

The project includes the unmodified official adult.data and adult.test source files and their accompanying dataset description. The files contain 32,561 and 16,281 records, respectively, totaling 48,842 observations. UCI specifies 14 input attributes and a binary income target [1]. Source files are cached locally for offline training, with an environment variable permitting an alternative data directory.

The numeric fields are age, fnlwgt, education_num, capital_gain, capital_loss, and hours_per_week. Categorical fields are workclass, education, marital_status, occupation, relationship, race, sex, and native_country. The target is encoded as 0 for income at or below USD 50,000 and 1 for income above that threshold. Trailing periods in official-test labels are removed before encoding. The question-mark marker is converted to a missing value.

Race and sex remain in the request schema and audit records but are excluded from classifier predictors. Sex is used for the reported fairness diagnostic. Removing explicit protected attributes does not remove information carried by correlated variables such as relationship, occupation, or marital status. Also, fnlwgt is used as an ordinary numeric predictor; the reported metrics are unweighted sample metrics rather than population-weighted estimates.

B. Splits and Leakage Prevention

A stratified split with random seed 42 partitions the official training file into 26,048 fitting records and 6,513 validation records. Both models use exactly this partition. The official test file is evaluated separately after fitting and is not used to estimate preprocessing parameters or decide admission thresholds. The saved drift baseline contains fitting inputs only. Table I summarizes the protocol.

All transformations are included in the persisted estimator pipeline. Numeric fields receive median imputation followed by standard scaling. Categorical predictor fields receive most-frequent imputation and one-hot encoding with unknown-category handling. Imputation values, scaling parameters, and category vocabularies are learned exclusively from fitting records. The strict API schema additionally rejects unsupported categories before serving.

TABLE I
DATA PARTITIONS AND THEIR PURPOSE

Partition	Records	Purpose
Fitting	26,048	Fit pipeline parameters
Validation	6,513	Quality / fairness admission
Official test	16,281	Final accuracy only

C. Models and Reproducibility

The v1 baseline uses logistic regression with max_iter=1000 and random_state=42. The v2 candidate uses a random forest with 100 estimators, min_samples_leaf=5, random_state=42, and n_jobs=1. Other estimator parameters retain their scikit-learn defaults. The comparison uses a fixed configuration rather than a hyperparameter search or repeated cross-validation study.

The recorded environment uses scikit-learn 1.7.2, NumPy 2.3.3, and pandas 2.3.2. A JSON manifest records the schema version adult-income-v1, split sizes, library versions, source-file hashes, fitting-baseline hash, artifact hashes, quality reports, and optional tracking identifiers. A SHA-256 hash binds a file to its recorded bytes; it is not an independent authenticity guarantee if both the artifact and manifest are replaced.

IV. SYSTEM ARCHITECTURE AND IMPLEMENTATION

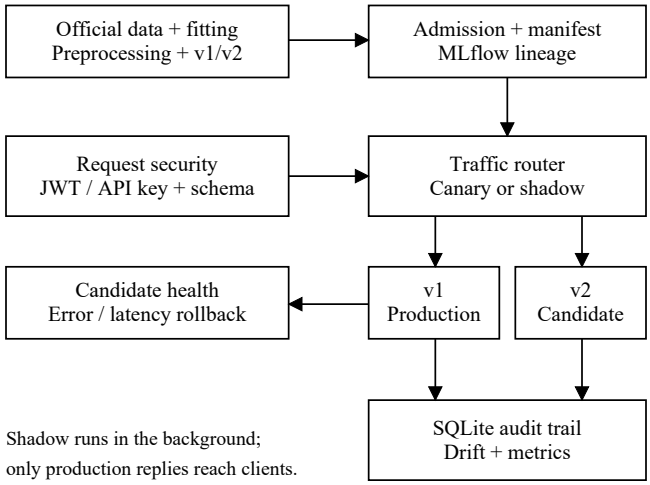


Fig. 1. Offline admission and online request governance. Canary and shadow are alternative routing modes.

A. Authenticated and Validated Serving

Flask exposes an authenticated prediction endpoint. Clients obtain a JWT through the login endpoint or supply a runtime-configured API key. Flask-Limiter restricts request frequency, while Pydantic enforces the Adult schema. Requests must contain the expected fields, reject additional fields, and respect strict numeric types and field-specific ranges. Boolean values and numeric strings are rejected for integer fields. Categorical values must match the configured vocabulary.

Only workclass, occupation, and native_country accept a missing marker or JSON null, corresponding to the documented missing categories. Nonfinite numeric values, unsupported categories, and oversized prediction payloads are rejected before model execution. These controls address malformed and out-of-domain requests; they do not establish robustness against all valid-input adversarial attacks. Demo credentials can be replaced through environment variables.

B. Admission and Integrity Checks

The runnable pre-deployment gate verifies the candidate artifact hash and schema, the source-data hashes, and the fitting-baseline hash. It then evaluates validation accuracy, per-class accuracy, demographic parity, stress handling, and repeatability. The API independently checks the model hash and schema before deserializing a configured artifact. Model files and their manifests must therefore come from a trusted source.

Admission requires validation accuracy of at least 0.80 and a demographic-parity gap no larger than 0.20, together with successful loading, stress handling, and repeatable predictions.

These are demonstration policy values, configurable through the admission CLI. Repeatability is checked by predicting the same probe ten times. Stress checks use a valid mixed-type record and variants of its numeric fields containing zero, large signed magnitudes, NaN, or infinity. Clean rejection of nonfinite stress input is accepted.

C. Canary, Shadow, and Rollback Controls

The default canary weights are 90 for v1 and 10 for v2. A weighted random selection chooses the serving model for each request; the observed fraction converges toward the configured share rather than enforcing an exact split in every batch. An authenticated runtime endpoint changes candidate weight and shadow settings without restarting the process, provided the loaded candidate has a passing stored gate result.

Shadow mode serves every live response from v1 and sends a configured fraction of inputs to v2. Candidate work runs through a single background executor with at most 32 outstanding shadow submissions. Saturation skips submission instead of holding up the production response. Separate candidate audit records share the live request identifier. The platform counts prediction disagreements; agreement is not a measurement of accuracy because online ground-truth income labels are unavailable.

Candidate rollback is enabled after at least 20 candidate requests. It activates when the candidate error rate exceeds 0.05 or the mean of up to the last 50 candidate latencies exceeds 500 ms. Subsequent requests return to v1, and further shadow submissions stop. Already queued work may finish. Production failures do not independently trigger candidate rollback. Resetting rollback clears the candidate observation window for a new assessment.

D. Audit, Drift, and Observability

SQLite stores prediction events through SQLAlchemy. Audit fields include timestamp, request identifier, client address, model version, validated inputs, prediction, latency, shadow flag, error indicator, and HTTP status. Rejected validated-path requests are also recorded. The audit trail is useful for local investigation but is not tamper-evident; audit write failures are suppressed to preserve inference availability.

Drift monitoring compares recent successful served requests with the fitting-data baseline. Shadow duplicates and rejected records are excluded. The default window is 100 records, with at least 30 records required before a result is reported. Each numeric feature uses a two-sample Kolmogorov-Smirnov test; categorical features use a contingency-table chi-square test. A feature is flagged when its p-value is below 0.05, and any flagged feature sets the overall drift indicator.

MLflow records pipeline artifacts, signatures, parameters, accuracy, gate status, dataset fingerprints, and optional registry versions. The registry was exercised locally for both pipelines, and a reloaded candidate produced a valid prediction. Prometheus exposition supplies traffic, errors, latency, rollback state, and configured weights. Docker Compose includes a generated authenticated scrape configuration and provisioned Grafana panels.

V. EVALUATION AND RESULTS

A. Classification and Fairness Metrics

Accuracy is the fraction of validation or test examples whose predicted binary label equals the recorded label. Per-class accuracy is the fraction classified correctly within each true class, equivalent to class recall. No claim of statistical significance is made from this single fixed split. Precision, F1, calibration, and ROC-AUC are not included in the saved experiment report.

For group g, define r_g as the number of positive predictions in group g divided by the number of evaluated records in that group. The demographic-parity diagnostic is DP_gap = |r_Male - r_Female|. It measures prediction-rate disparity across the dataset's recorded binary sex groups, not equal opportunity, causal fairness, or individual fairness. The 0.20 limit is a chosen engineering admission policy rather than a legal or universal fairness standard.

B. Measured Model Outcomes

TABLE II
MEASURED MODEL RESULTS (SEED 42)

Model	Val. acc.	Test acc.	DP gap	Gate
v1: LR	85.57%	85.11%	0.1715	Pass
v2: RF	86.57%	86.32%	0.1762	Pass

Table II reports the saved model outcomes. The random forest reaches 86.57% validation accuracy and 86.32% official-test accuracy. Logistic regression reaches 85.57% and 85.11%, respectively. The candidate therefore improves official-test accuracy by 1.21 percentage points in this experiment. Both models pass the configured accuracy and demographic-parity requirements.

The candidate's validation positive-prediction rates are 0.0700 for Female and 0.2462 for Male, producing a gap of 0.1762. The baseline rates are 0.0834 and 0.2549, producing a gap of 0.1715. Candidate class recall is 94.64% for the <=50K class and 61.10% for the >50K class. Baseline recall is 93.31% and 61.16%, respectively. Thus, improved overall accuracy is accompanied by a slightly larger parity gap and does not improve positive-class recall in this split.

These outcomes illustrate why a single aggregate score is insufficient for release decisions. A configurable gate makes the selected policy explicit and reproducible, while per-group and per-class measurements reveal trade-offs that remain after admission. The gate passing result should be interpreted within its measured scope.

C. Functional Verification

TABLE III
VERIFICATION EVIDENCE AND SCOPE

Verification	Observed outcome
Automated suite	44 tests pass locally
Candidate admission	Pass; integrity verified
Stress probes	0 candidate stress failures
MLflow registry	Both models registered locally
Reloaded MLflow model	Valid candidate prediction
Compose / workflows	Configuration checks pass
Container runtime	Not run: daemon unavailable
Remote deployment	Not executed

The local automated suite contains 44 passing tests. Verification covers valid Adult prediction responses, required authentication, API-key access, strict types and feature bounds, unsupported categories, missing-value imputation, official dataset splits, and model metadata. Governance tests check candidate quality, fairness-based rejection under a stricter threshold, deterministic prediction, and rejection of a corrupted artifact before deserialization.

Routing tests exercise approximate canary shares, production-only shadow responses, candidate rollback, and isolation of production failures from candidate rollback decisions. Integration checks demonstrate runtime weight changes, drift computation after sufficient Adult audit events, separate served and shadow audit rows, and a blocked shadow candidate that does not prevent the production call from returning. These tests demonstrate implemented behavior; no sustained load or availability benchmark was conducted.

The admission CLI passes for v2 with zero stress failures and verified integrity. Local MLflow tracking, model registration, and prediction after model reload were also verified. Docker Compose configuration and embedded workflow scripts were checked. The Docker daemon was unavailable during validation, so end-to-end container execution and dashboard scraping were not observed. GitHub Actions was configured but was not executed on a remote runner as part of this local evaluation.

VI. DEPLOYMENT WORKFLOW AND LIMITATIONS

The local sequence is dependency installation, pipeline training, admission evaluation, automated tests, and API startup. GitHub Actions automates training, MLflow logging, testing, and candidate admission. A manually triggered canary workflow validates traffic weights and uploads a release bundle containing models, manifest, baseline, configuration, and tracking data. It prepares deployable evidence rather than performing an unconfigured remote rollout.

The Compose API uses one worker with threads because traffic weights, counters, and rollback state are process-local. Runtime routing changes are not persisted to the configuration file. Multiple replicas would require coordinated state, a shared model registry, and consistent rollout policy. Replacing model binaries still requires a separate loading or deployment mechanism. Zero downtime in this work refers to traffic switching between already loaded models; it is not an availability guarantee during host failures or software replacement.

Security evaluation is limited to the implemented API boundary and regression suite. JWT and API-key authorization currently share access to protected operational endpoints; granular administrator roles, distributed rate-limit storage, transport security termination, secret rotation, and immutable audit storage require further work. Validated records include demographic attributes, so retention limits and access controls would be needed for deployment beyond the academic demonstration.

Distribution tests have limitations. Running 14 uncorrected tests increases the chance of at least one false alarm, while sparse categorical counts can weaken chi-square approximations. No effect-size threshold, multiple-testing adjustment, alert persistence policy, automatic retraining, or label-based concept-drift analysis is implemented. Historical Adult data and a single binary group comparison also limit the fairness conclusions that can be drawn.

VII. CONCLUSION AND FUTURE WORK

This project demonstrates a locally reproducible MLOps governance workflow around UCI Adult income classification. It connects fitted preprocessing pipelines, explicit admission policy, authenticated serving, controlled canary and shadow exposure, candidate rollback, and data-to-prediction traceability. The candidate's 86.32% official-test accuracy and the passing 44-test suite provide evidence for the model and functional behavior actually evaluated.

Future work should introduce shared rollout state, stronger authorization, tamper-evident auditing, delayed-label feedback, and repeated performance evaluation. Fairness analysis should extend to additional protected groups, intersectional slices, and alternative criteria. Drift monitoring would benefit from effect sizes, corrected significance testing, and persistence-aware alerts. Container execution, sustained-load testing, and an explicitly configured remote deployment target are necessary before making broader availability or production-readiness claims.

REFERENCES

[1] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, doi: 10.24432/C5XW20. [Online].

[2] D. Sculley et al., "Hidden technical debt in machine learning systems," in Advances in Neural Information Processing Systems, vol. 28, 2015. [Online].

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011. [Online].

[4] F. Ding, M. Hardt, J. Miller, and L. Schmidt, "Retiring Adult: New datasets for fair machine learning," in Advances in Neural Information Processing Systems, vol. 34, 2021. [Online].